VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

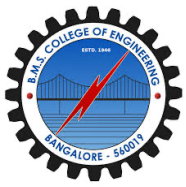
LAB REPORT
on

Object Oriented Java programing(23CS3PCOOJ)

Submitted by

Varsha M Gowda(1WA23CS033)

In partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



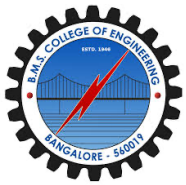
B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019

B.M.S. College of Engineering,

Bull Temple Road, Bangalore-560019

(Affiliated To Visvesvaraya Technological University, Belgaum)

Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled "Object Oriented Java Programming (23CSPCOOJ)" carried out by **Varsha M Gowda(1WA23CS033),** who is bonafide student of **B.M.S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements in respect of a Database Management Systems(23CS3PCDBM) work prescribed for the said degree.

Lab faculty in charge Name Assistant Professor Department of CSE, BMSCE	Dr. Joythi S Nayak Professor & HOD Department of CSE, BMSCE

Index

Sl.No	Date	Experiment Title	Page No.
1.	09/10/24	Quadratic Equation	4
2.	09/10/24	Calculate SGPA	5
3.	23/10/24	N book objects	7
4.	23/10/24	Area of the given shape	9
5.	30/10/24	Bank Account	11
6.	30/10/24	Packages	14
7.	13/11/24	Expectations	16
8.	20/11/24	Threads	18
9.	4/12/24	Interface to perform integer division	19
10.	4/12/24	Inter process Communication and Deadlock	21

Experiment 1

```
import java.io.*;
import java.util.Scanner;

class Quad{
	public static void main(String args[]){
		double a,b,c,s1,s2;
		Scanner sc = new Scanner(System.in);
		System.out.println("enter a b c of quadratic eqtn");
		a = sc.nextDouble();
		b = sc.nextDouble();
		c = sc.nextDouble();
		if(a==0){
			System.out.print("INVALID EQTN");
			return;
		}
		double d= b*b-4*a*c;
		if (d>0){
			System.out.println("real and distinct roots");
			s1=(-b + Math.sqrt(d))/(2*a);
			s2=(-b - Math.sqrt(d))/(2*a);
			System.out.print("roots are "+s1+" "+s2);
		}else if(d==0){
			System.out.println("real and equal roots");
			s1= -b/(2*a);
			System.out.print("root: "+s1);
		}else{
			s1= -b/(2*a);
			s2= Math.sqrt(-d)/(2*a);
			System.out.println("imaginary roots");
			System.out.println("root 1: "+s1+" +i "+s2);
			System.out.println("root 1: "+s1+" -i "+s2);
		}
	}
}
```

OUTPUT:

```
Enter values of a, b, and c:
1 2 1
The equation has equal roots: x1 = -1.0 x2= -1.0
```

```
Enter values of a, b, and c:
2 3 5
The equation has imaginary roots. There is no real solution.
```

```
Enter values of a, b, and c:

The equation has two real solutions: x1 = -1.0, x2 = -2.0
```

Experiment 2

```
import java.io.*;
import java.util.Scanner;

class    StudentD{
String usn, name;
int[] m1=new int[10],m2=new int[10];
int[] c1 = new int[10], c2 = new int[10];
int sub;

void accept(String n,String u, int s){

this.usn=u;
this.name=n;
this.sub=s;
Scanner sc = new Scanner(System.in);
System.out.println("enter details of sem1");
for(int i=0;i<s;i++){
        System.out.print("Enter marks for subject " + (i + 1) + ": ");
        m1[i] = sc.nextInt();
        System.out.print("Enter credits for subject " + (i + 1) + ": ");
        c1[i] = sc.nextInt();
}
System.out.println("Enter details for Semester 2:");
for (int i = 0; i < sub; i++) {
        System.out.print("Enter marks for subject " + (i + 1) + ": ");
        m2[i] = sc.nextInt();
        System.out.print("Enter credits for subject " + (i + 1) + ": ");
        c2[i] = sc.nextInt();
}

}

void display() {

System.out.println("USN: " + usn);
System.out.println("Name: " + name);

}

double sgpa(int sem){

double totalm=0,totalc=0;
if (sem==1){
for(int i=0;i<sub;i++){
totalm+=m1[i]*c1[i];
totalc+=c1[i];
}
```

```
}
else if (sem == 2) {
for (int i = 0; i < sub; i++) {
totalm += m2[i] * c2[i];
totalc += c2[i];
}
}
if (totalc == 0) return 0;
return totalm / (totalc*10);

}


double calcCgpa() {
double sgpaSem1 = sgpa(1);
double sgpaSem2 = sgpa(2);
return (sgpaSem1 + sgpaSem2) / 2;
}
}

public class Student {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
System.out.print("Enter the number of subjects: ");
int sub = sc.nextInt();

Student student = new Student();
student.accept("Thanu","1WA23CS019", sub);
student.display();
double sgpaSem1 = student.sgpa(1);
double sgpaSem2 = student.sgpa(2);
System.out.println("SGPA for Semester 1: " + sgpaSem1);
System.out.println("SGPA for Semester 2: " + sgpaSem2);
double cgpa = student.calcCgpa();
System.out.println("CGPA: " + cgpa);
sc.close();
}
}
```

OUTPUT:

```
Enter USN: 1BM23CS003
Enter Name: Alice
Enter number of subjects: 5
Enter credits for subject 1: 4
Enter marks for subject 1: 78
Enter credits for subject 2: 4
Enter marks for subject 2: 88
Enter credits for subject 3: 3
Enter marks for subject 3: 66
Enter credits for subject 4: 3
Enter marks for subject 4: 45
Enter credits for subject 5: 2
Enter marks for subject 5: 100
Student Details:
USN: 1BM23CS003
Name: Alice
Subject-wise Details:
Subject 1: Credits = 4, Marks = 78
Subject 2: Credits = 4, Marks = 88
Subject 3: Credits = 3, Marks = 66
Subject 4: Credits = 3, Marks = 45
Subject 5: Credits = 2, Marks = 100
SGPA: 7.75
```

Experiment 3

```
import java.util.Scanner;
class Book {
   String name;
   String author;
   double price;
   int pages;
public Book(String name, String author, double price, int pages) {
      this.name = name;
      this.author = author;
      this.price = price;
      this.pages = pages;
   }
public String toString(){
return "name="+name+"\n"+"author="+author+"\n"+"price="+price+"\n"+"pages="+pages;
}
}
public class BookInfo {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter the number of books: ");
      int n = sc.nextInt();
      sc.nextLine();
      Book[] books = new Book[n];
      for (int i = 0; i < n; i++) {
         System.out.println("enter details for book " + (i + 1) + ":");
         System.out.print("enter the name of book: ");
         String name = sc.nextLine();
         System.out.print("enter the name of author: ");
         String author = sc.nextLine();
         System.out.print("enter the price: ");
         double price = sc.nextDouble();
         sc.nextLine();
         System.out.print("enter the number of pages: ");
         int pages = sc.nextInt();
         sc.nextLine();
         books[i] = new Book(name, author, price, pages);
      }

      for (int i = 0; i < n; i++) {
         System.out.println("Details of book " + (i + 1) + ":");
         System.out.println(books[i].toString());

      }

      sc.close();
   }
}
```

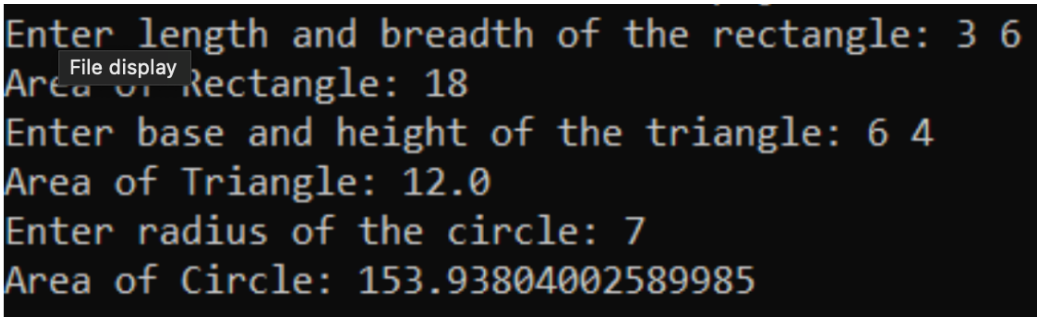
OUTPUT:

```
Enter number of books:
3
Book 1:
Enter name of Book: Brave New World
Enter Author name: Aldous Huxley
Enter price: 500
Enter number of pages: 250
Book 2:
Enter name of Book: Merchant of Venice
Enter Author name: Shakespeare
Enter price: 750
Enter number of pages: 400
Book 3:
Enter name of Book: The Da Vinci Code
Enter Author name: Dan Brown
Enter price: 999
Enter number of pages: 689
Book details:
Book name: Brave New World
Author name: Aldous Huxley
Price: 500.0
Number of pages: 250
Book name: Merchant of Venice
Author name: Shakespeare
Price: 750.0
Number of pages: 400
Book name: The Da Vinci Code
Author name: Dan Brown
Price: 999.0
Number of pages: 689
```

Experiment 4

```
import java.util.*;
abstract class Figure{
double dim1;
double dim2;
Figure(double a,double b)
{
dim1=a;
dim2=b;
}
abstract double area();
}
class rectangle extends Figure{
rectangle(double a,double b){super(a,b);}
double area()
{
System.out.println("inside area of rectangle:");
return dim1*dim2;
}
}
class triangle extends Figure{
triangle(double a,double b){super(a,b);}
double area()
{
System.out.println("inside area of triangle:");
return (dim1*dim2)/2;
}
}
class circle extends Figure{
circle(double a){super(a,a);}
double area()
{
System.out.println("inside area of circle:");
return Math.PI*(dim1*dim1);
}
}
class AbstractArea{
public static void main(String [] args){
rectangle r=new rectangle(10,20);
triangle t=new triangle(3,5);
circle c=new circle(9);
Figure figref;
figref=r;
System.out.println("area is:"+figref.area());
figref=t;
System.out.println("area is:"+figref.area());
figref=c;
System.out.println("area is:"+figref.area());
```

}}
OUTPUT:


Enter length and breadth of the rectangle: 3 6
Area of Rectangle: 18
Enter base and height of the triangle: 6 4
Area of Triangle: 12.0
Enter radius of the circle: 7
Area of Circle: 153.93804002589985

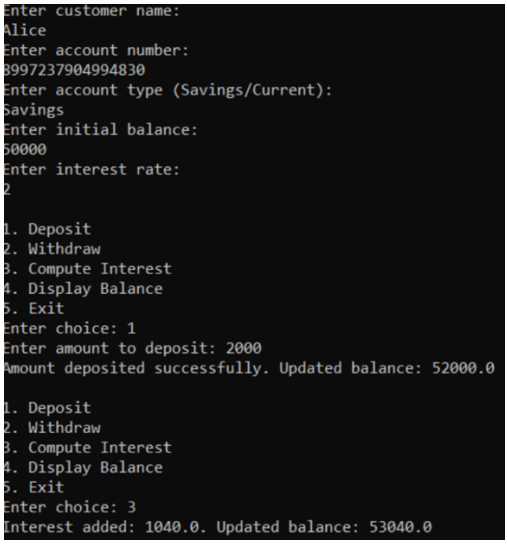
Experiment 5

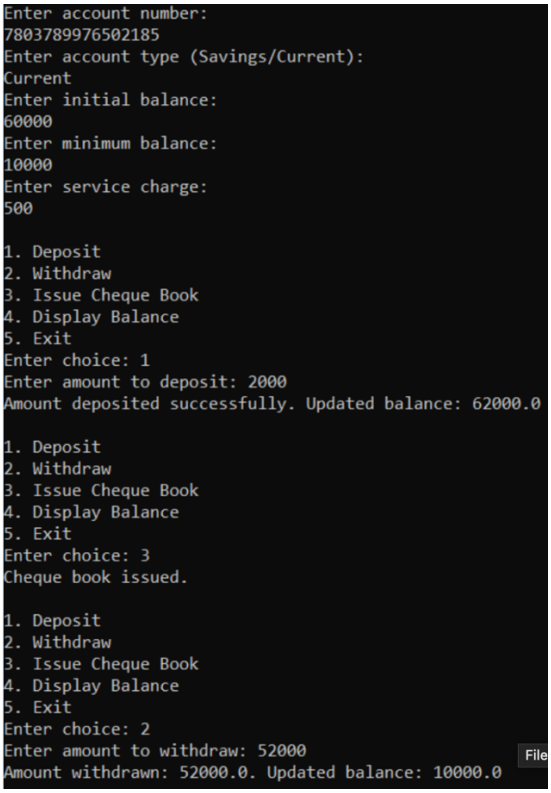
```
import java.util.Scanner;
class Account {
   protected String customerName;
   protected String accountNumber;
   protected String accountType;
   protected double balance;

   public Account(String customerName, String accountNumber, String accountType, double
initialBalance) {
      this.customerName = customerName;
      this.accountNumber = accountNumber;
      this.accountType = accountType;
      this.balance = initialBalance;
   }

   public void deposit(double amount) {
      if (amount > 0) {
         balance += amount;
         System.out.println("Amount deposited successfully.");
      } else {
         System.out.println("Invalid deposit amount.");
      }
   }

   public void displayBalance() {
      System.out.println("Account Balance: " + balance);
   }
}

class SavAcct extends Account {
   private double interestRate;

   public SavAcct(String customerName, String accountNumber, double initialBalance,
double interestRate) {
      super(customerName, accountNumber, "Savings", initialBalance);
      this.interestRate = interestRate;
   }

   public void computeAndDepositInterest() {
      double interest = balance * interestRate / 100;
      balance += interest;
      System.out.println("Interest of " + interest + " has been deposited.");
   }

   public void withdraw(double amount) {
      if (amount > 0 && amount <= balance) {
         balance -= amount;
```

```
            System.out.println("Withdrawal successful. New balance: " + balance);
        } else {
            System.out.println("Invalid withdrawal amount or insufficient balance.");
        }
    }
}

class CurAcct extends Account {
    private static final double MINIMUM_BALANCE = 500.0;
    private static final double PENALTY = 50.0;

    public CurAcct(String customerName, String accountNumber, double initialBalance) {
        super(customerName, accountNumber, "Current", initialBalance);
    }

    public void checkAndImposePenalty() {
        if (balance < MINIMUM_BALANCE) {
            balance -= PENALTY;
            System.out.println("Penalty of " + PENALTY + " imposed due to insufficient
balance.");
        }
    }

    public void withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
            System.out.println("Withdrawal successful. New balance: " + balance);
            checkAndImposePenalty();
        } else {
            System.out.println("Invalid withdrawal amount or insufficient balance.");
        }
    }
}

public class Bank {
    public static void main(String[] args) {
        SavAcct savingsAccount = new SavAcct("Alice", "SA12345", 1000.0, 5.0);
        CurAcct currentAccount = new CurAcct("Bob", "CA54321", 600.0);

        // Savings account operations
        System.out.println("Savings Account Operations:");
        savingsAccount.deposit(500);
        savingsAccount.displayBalance();
        savingsAccount.computeAndDepositInterest();
        savingsAccount.withdraw(300);
        savingsAccount.displayBalance();

        // Current account operations
        System.out.println("\nCurrent Account Operations:");
        currentAccount.deposit(200);
```

```
        currentAccount.displayBalance();
        currentAccount.withdraw(100);
        currentAccount.displayBalance();
        currentAccount.withdraw(700);
        currentAccount.displayBalance();
    }
}
```

OUTPUT:





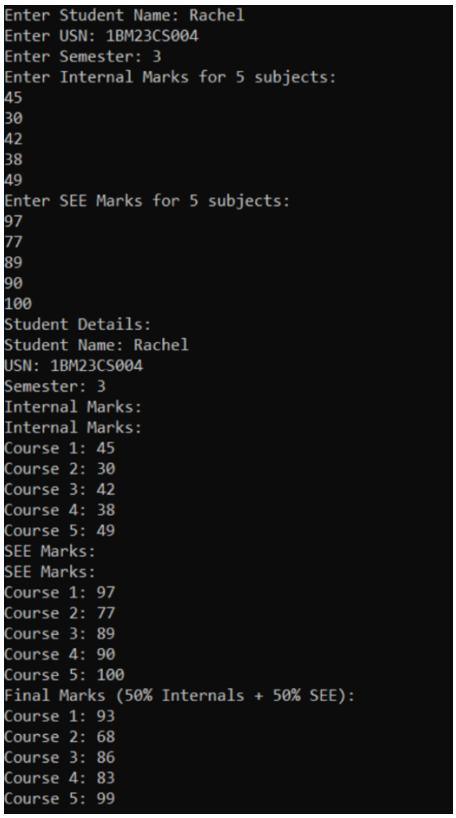
Experiment 6

```
import CIE.*;
import SEE.*;
import java.util.Scanner;

class Marks{
public static void main(String args[]){
Scanner sc=new Scanner(System.in);

System.out.println("Enter internal marks for 5 courses (out of 50): ");
int internalMarks[] = new int[5];
for (int j=0;j<5;j++)
{
System.out.print("Course " + (j + 1) + " internal marks: ");
internalMarks[j] = sc.nextInt();
}
Internals im = new Internals(internalMarks);
System.out.println("Enter SEE marks for 5 courses (out of 50): ");
int seeMarks[] = new int[5];
for (int j=0;j<5;j++)
{
System.out.print("Course "+(j + 1)+" SEE marks: ");
seeMarks[j] = sc.nextInt();
}
Externals ext=new Externals("Thanu","yy78",2,seeMarks);
System.out.println("\nFinal Marks for " + ext.name + ":" + ext.usn + ":for:");
System.out.println("Semester: " + ext.sem);
for(int x=0;x<5;x++){
int total=0;
total=im.m[x]+ext.m[x];
System.out.println("subject "+(x+1)+":  "+total);
}
}
}
```

```
package CIE;

public class Internals{
        public int[] m = new int[5];
        public Internals(int[] marks) {
        for (int i = 0; i < 5; i++) {
                m[i] = marks[i];
        }
        }
}
```

```
package CIE;

public class Student{
        public String usn;
        public String name;
        public int sem;
        public Student(String n, String u,int s){
        usn=u;
        name=n;
        sem=s;
        }
}

package SEE;
import CIE.Student;

public class Externals extends Student{
public int[] m = new int[5];
public Externals(String name, String usn, int sem,int seeMarks[]){
super(name,usn,sem);
for (int i = 0; i < 5; i++)
{
this.m[i] = seeMarks[i];
}
}
}
```

OUTPUT:



Experiment 7

```
class WrongAgeException extends Exception {
   public WrongAgeException(String message) {
      super(message);
   }
}

class Father {
   protected int age;

   public Father(int age) throws WrongAgeException {
      if (age < 0) {
         throw new WrongAgeException("Father's age cannot be negative!");
      }
      this.age = age;
   }
}

class Son extends Father {
   private int sonAge;

   public Son(int fatherAge, int sonAge) throws WrongAgeException {
      super(fatherAge);
      if (sonAge < 0) {
         throw new WrongAgeException("Son's age cannot be negative!");
      }
      if (sonAge >= fatherAge) {
         throw new WrongAgeException("Son's age cannot be greater than or equal to Father's
age!");
      }
      this.sonAge = sonAge;
   }

   public void displayAges() {
      System.out.println("Father's Age: " + age);
      System.out.println("Son's Age: " + sonAge);
   }
}

public class ExceptionHandling {
   public static void main(String[] args) {
      try {
         Father father = new Father(40);
         Son son = new Son(40, 20);
         son.displayAges();
         Father invalidFather = new Father(-5);
      } catch (WrongAgeException e) {
         System.out.println("Exception: " + e.getMessage());
```

```
        }
        try {
            Son invalidSon = new Son(30, 35);
        } catch (WrongAgeException e) {
            System.out.println("Exception: " + e.getMessage());
        }
    }
}
```

OUTPUT:

```
Enter Father's Age:
45
Enter Son's Age:
65
Father's age: 45
SonOlderThanFatherException: Son's age cannot be greater than or equal to Father's age.
```

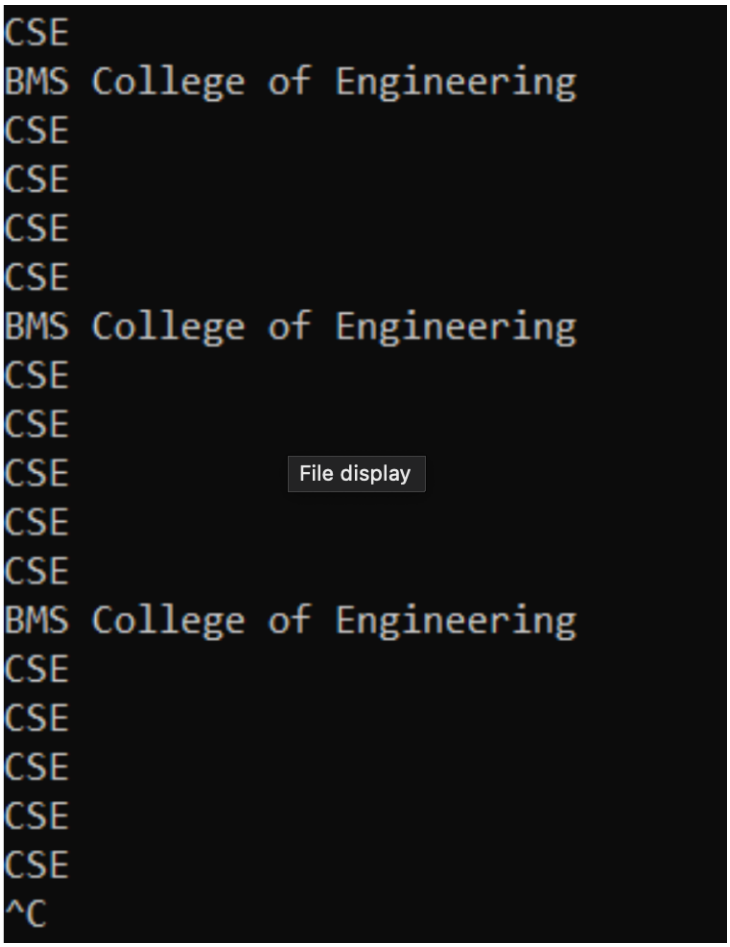
```
Enter Father's Age:
-20
Enter Son's Age:
5
WrongAgeException: Age cannot be negative.
```

```
Enter Father's Age:
55
Enter Son's Age:
20
Father's age: 55
Son's age: 20
```

Experiment 8

```
class CollegeThread extends Thread {
   public void run() {
      try {
         for (int i = 0; i < 5; i++) {
            System.out.println("BMS College of Engineering");
            Thread.sleep(10000);
         }
      } catch (InterruptedException e) {
         System.out.println("CollegeThread interrupted.");
      }
   }
}

class CSEThread extends Thread {
   public void run() {
      try {
         for (int i = 0; i < 25; i++) {
            System.out.println("CSE");
            Thread.sleep(2000);
         }
      } catch (InterruptedException e) {
         System.out.println("CSEThread interrupted.");
      }
   }
}

public class ThreadExample {
   public static void main(String[] args) {
      CollegeThread collegeThread = new CollegeThread();
      CSEThread cseThread = new CSEThread();

      collegeThread.start();
      cseThread.start();
   }
}
```

OUTPUT:

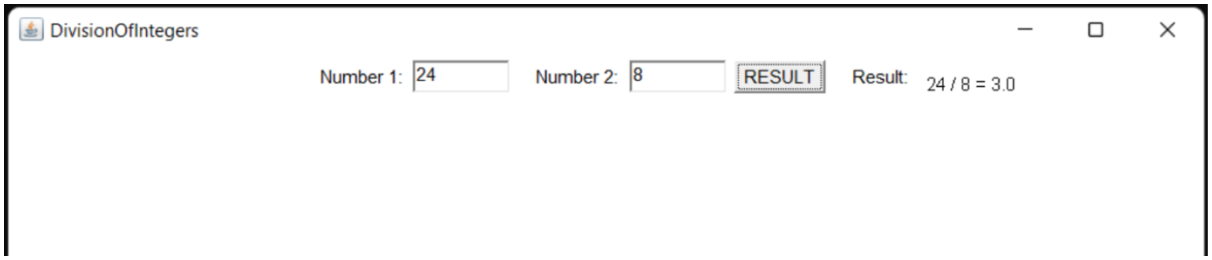

CSE
BMS College of Engineering
CSE
CSE
CSE
CSE
BMS College of Engineering
CSE
CSE
CSE
File display
CSE
CSE
BMS College of Engineering
CSE
CSE
CSE
CSE
CSE
^C

Experiment 9

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class IntDiv extends JFrame {
   private JTextField txtNum1, txtNum2, txtResult;
   private JButton btnDivide;

   public IntDiv() {
      setTitle("Integer Division Calculator");
      setSize(400, 250);
      setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
      setLayout(new GridLayout(4, 2, 10, 10));

      initializeComponents();

      addComponentsToFrame();

      setLocationRelativeTo(null);
   }

   private void initializeComponents() {

      JLabel lblNum1 = new JLabel("Number 1:");
      JLabel lblNum2 = new JLabel("Number 2:");
      JLabel lblResult = new JLabel("Result:");

      txtNum1 = new JTextField();
      txtNum2 = new JTextField();
      txtResult = new JTextField();
      txtResult.setEditable(false);

      btnDivide = new JButton("Divide");
      btnDivide.addActionListener(new ActionListener() {
         @Override
         public void actionPerformed(ActionEvent e) {
            performDivision();
         }
      });
   }

   private void addComponentsToFrame() {
      add(new JLabel("Number 1:"));
      add(txtNum1);
      add(new JLabel("Number 2:"));
      add(txtNum2);
      add(new JLabel("Result:"));
```

```
        add(txtResult);
        add(new JLabel());
        add(btnDivide);
    }

    private void performDivision() {
        try {
            int num1 = Integer.parseInt(txtNum1.getText());
            int num2 = Integer.parseInt(txtNum2.getText());
            int result = num1 / num2;
            txtResult.setText(String.valueOf(result));

        } catch (NumberFormatException nfe) {
            JOptionPane.showMessageDialog(
                this,
                "Please enter valid integers!",
                "Input Error",
                JOptionPane.ERROR_MESSAGE
            );
        } catch (ArithmeticException ae) {
            JOptionPane.showMessageDialog(
                this,
                "Cannot divide by zero!",
                "Arithmetic Error",
                JOptionPane.ERROR_MESSAGE
            );
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(new Runnable() {
            public void run() {
                new IntDiv().setVisible(true);
            }
        });
    }


}
```

OUTPUT:



Experiment 10

```
class AA {
  synchronized void foo(BB b) {
        String name = Thread.currentThread().getName();
        System.out.println(name + " entered A.foo");
        try {
              Thread.sleep(1000);
        } catch(Exception e) {
              System.out.println("A Interrupted");
        }
        System.out.println(name + " trying to call B.last()");
        b.last();
  }
  synchronized void last() {
        System.out.println("Inside A.last");
  }
}

class BB {
  synchronized void bar(AA a) {
    String name = Thread.currentThread().getName();
        System.out.println(name + " entered BB.bar");
        try {
              Thread.sleep(1000);
        } catch(Exception e) {
              System.out.println("B Interrupted");
        }
        System.out.println(name + " trying to call A.last()");
        a.last();
   }
   synchronized void last() {
        System.out.println("Inside A.last");
   }
}

class Deadlock implements Runnable {
        AA a = new AA();
        BB b = new BB();
        Deadlock() {
              Thread.currentThread().setName("MainThread");
              Thread t = new Thread(this, "RacingThread");
              t.start();
              a.foo(b); // get lock on a in this thread.
              System.out.println("Back in main thread");
        }
        public void run() {
              b.bar(a); // get lock on b in other thread.
              System.out.println("Back in other thread");
```

```
	}
	public static void main(String args[]) {
		new Deadlock();
	}
}

//INTERPROCESS COMMUNICATION

class Q {
	int n;
	synchronized int get() {
		System.out.println("Got: " + n);
		return n;
	}
	synchronized void put(int n) {
		this.n = n;
		System.out.println("Put: " + n);
	}
}

class Producer implements Runnable {
	Q q;
	Producer(Q q) {
		this.q = q;
		new Thread(this, "Producer").start();
	}
	public void run() {
		int i = 0;
		while(true) {
			q.put(i++);
		}
	}
}

class Consumer implements Runnable {
	Q q;
	Consumer(Q q) {
		this.q = q;
		new Thread(this, "Consumer").start();
	}
	public void run() {
		while(true) {
			q.get();
		}
	}
}

class Deadlock {
	public static void main(String args[]) {
		Q q = new Q();
```

```
                new Producer(q);
                new Consumer(q);
                System.out.println("Press Control-C to stop.");
        }
}
```

OUTPUT: